

Machine Learning in Sequences

Séance de TP 1

Hidden Markov Models (HMM) are generative models

1- Generate synthetic data with the trained HMM

- Write a method that exploits the generative Hidden Markov Model property to generate synthetic digits images of size 28 x 28. The trained models are made available in a specific folder following the naming

```
def GenerateSamples(self, N_samples, length):  
    # Initialize arrays to store generated states and observations.  
    seq_states = np.zeros((N_samples, length), dtype=int)  
    seq_observations = np.zeros((N_samples, length), dtype=int)  
  
    # Create a random number generator using numpy's default RNG  
    generator = np.random.default_rng()  
  
    Intervals_A = np.roll(np.cumsum(np.exp(self.log_A), axis=1), 1)  
    Intervals_A[:, 0] = 0 # Set the first column of Intervals_A to 0  
  
    Intervals_B = np.roll(np.cumsum(np.exp(self.log_B), axis=1), 1)  
    Intervals_B[:, 0] = 0 # Set the first column of Intervals_B to 0  
  
    n = 0 # Sample counter  
  
    # Loop over the desired number of samples (N_samples)  
    while n < N_samples:  
        # Generate an array of random numbers for the state sequence.  
        draw_state = generator.random((length  
  
        # Loop over the time steps to generate the states.  
        for t in range(1, length - 1):  
            # Get the cumulative distribution (CDF) for the previous  
state  
            I = Intervals_A[seq_states[n, t - 1], :]  
            test = I[I < draw_state[t]]  
            seq_states[n, t] = np.size(test) - 1  
            if seq_states[n, length - 2] > self.n_states - 3:  
                seq_states[n, length - 1] = self.n_states - 1
```

```

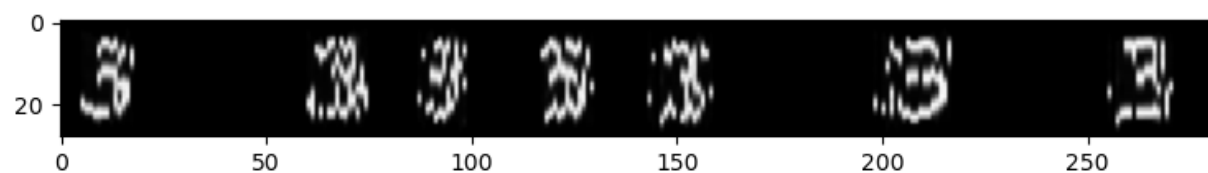
# Generate observations based on the final states
draw_obs = generator.random((length,))
for t in range(length - 1):
    I = Intervals_B[seq_states[n, t], :]
    test = I[I < draw_obs[t]]
    # Set the observation for this time step
    seq_observations[n, t] = np.size(test) - 1
# Increment the sample counter
n += 1

# Return the generated sequences of states and observations
return seq_states, seq_observations

```

b. How do you find the quality of the generated samples?

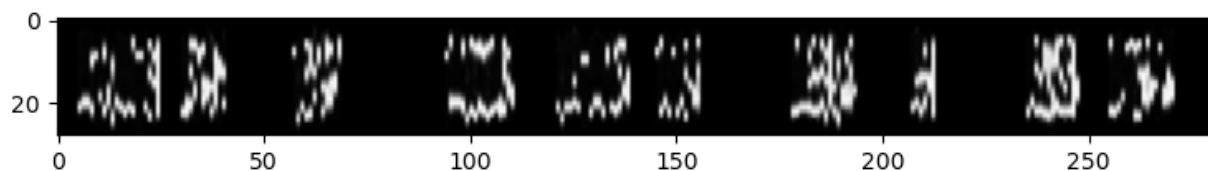
n_clusters = 100, n_states = 15



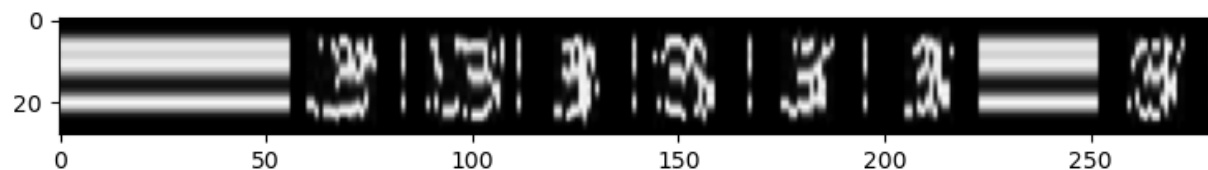
n_clusters = 100, n_states = 10



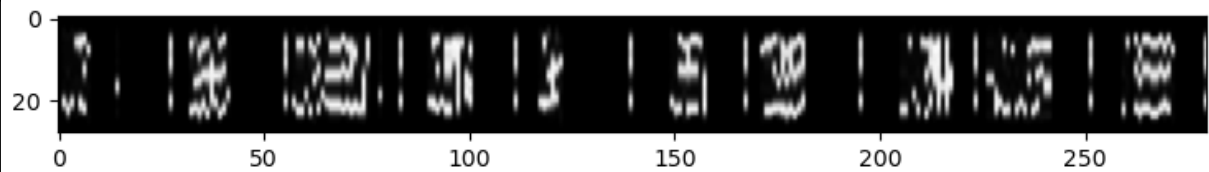
n_clusters = 100, n_states = 5



n_clusters = 50, n_states = 15



n_clusters = 50, n_states = 5



We can see from the generated sequence of images that examples with a smaller number of clusters and states present lower sample quality. This translates into representations with a different morphology than the expected digits, or incomplete or fractured representations.

Increasing the number of states allows the HM to model more complex patterns, but it also appears to increase the constraints of the generation condition, resulting in the generation of gaps or constant horizontal lines instead of digits.

c. How would you qualify the imperfection of the generated data ?

The samples generated present structural imperfections (fragmentation and distortion) in its formation for all the parameters tested. Additionally, for models trained with a lower value of states, some noise can be seen on the images.

d. What could be the reasons of the quality observed?

The low-cluster seems to limit the generating function where insufficient data forces the model to approximate complex shapes with inadequate representations, leading to fragmented outputs.

Also, higher values of states appears to make the sampling condition harder to satisfy, resulting in either repetitive horizontal patterns (when few sequences meet the condition) or blank images (when none do).

2- Classify the synthetic data using Viterbi

- a. Write a method that computes the log likelihood of the best hidden state path using the Viterbi algorithm, using the following interface.

```
def Viterbi(self, obs_seq):
    T = obs_seq.shape[0]
    Viterbi_lattice = np.zeros((T, self.n_states))
    work_buffer = np.zeros(self.n_states)

    Viterbi_lattice[0, :] = self.log_Pi + self.log_B[:, obs_seq[0]]
    for t in range(1, T):
        for j in range(self.n_states):
            work_buffer = Viterbi_lattice[t-1, :] + self.log_A[:, j]
            Viterbi_lattice[t, j] = np.max(work_buffer) +
                                    self.log_B[j, obs_seq[t]]

    Viterbi_lattice[T-1, :] += self.log_Pf

    LL_best_path = np.max(Viterbi_lattice[T-1, :])

    return LL_best_path
```

- b. Then, compute the 10 log likelihood scores of the ten models for each generated observation sequence and vote for the best model that can be associated.

```
# Compute Loglikelihood of the best path with Viterbi
for c in range(N_classes):
    LL_scores[c] = models[c].Viterbi(seq_observations[sample])

predicted_class = np.argmax(LL_scores)
print("\nLL_scores", LL_scores)
print("Classe:", classe, "meilleur model :", predicted_class,
      "LL:", np.max(LL_scores))
```

```
LL_scores [-4776.61714264 -67.69040838 -687.57522496 -1859.70987002
 -94.16917776 -247.07016951 -154.938093 -345.03269836 -1444.36220553 -
659.7670265]
```

```
Classe: 3 meilleur model : 1 LL: -67.6904083816359
```

```
LL_scores [-247.54012338 -109.79209826 -78.36262733 -47.32686062 -
242.40422409 -85.21376578 -220.69829217 -132.90200088 -96.29928267 -
125.95582334]
```

Classe: 3 **meilleur model** : 3 LL: -47.326860617289775

We can see that the function is able to calculate the 10 log likelihood scores of each class for every digit in sample and choose the highest value as the predicted class. The first result is the misclassification of an empty digit, while the second one is a good classification of a digit with satisfactory quality.

c. Compute the Character Recognition Rate (CRR) and the Character Error Rate (CER) over the generated data.

```
if predicted_class == classe:
    correct_characters += 1

# Increase the number of characters
total_characters += 1

# Check if the class prediction is incorrect to calculate the
CER
if predicted_class != classe:
    total_errors += 1

CRR = correct_characters / total_characters
CER = total_errors / total_characters

print(f"Character Recognition Rate (CRR): {CRR * 100:.2f}%")
print(f"Character Error Rate (CER): {CER * 100:.2f}%")
```

```
n_clusters = 100, n_states = 15
Character Recognition Rate (CRR): 91.00%
Character Error Rate (CER): 9.00%
```

```
n_clusters = 100, n_states = 10
Character Recognition Rate (CRR): 84.00%
Character Error Rate (CER): 16.00%
```

n_clusters = 100, n_states = 5

Character Recognition Rate (CRR): 89.00%

Character Error Rate (CER): 11.00%

n_clusters = 50, n_states = 15

Character Recognition Rate (CRR): 14.00%

Character Error Rate (CER): 86.00%

n_clusters = 50, n_states = 5

Character Recognition Rate (CRR): 39.00%

Character Error Rate (CER): 61.00%

As expected, the Viterbi performed better in samples generated with a higher number of states, since they present a better quality. It is important to mention that the samples that consist in empty spaces or horizontal lines are the greatest cause of misclassification.

3- Classify the real data using Viterbi

- a. *Use the MNIST test dataset to test the recognition performance on the real data. Compute the CRR and the CER.*

```
# Evaluate on MNIST test data
print("Evaluating on MNIST test data...")
true_labels = []
predicted_labels = []

total_test_characters = min(1000, len(X_test))
correct_test_characters = 0
total_test_errors = 0

LL_scores_test = np.zeros((N_classes,))

for i in tqdm(range(min(1000, len(X_test)))): # Limit to 1000 samples
    observation_sequence = X_test_discrete[i]
    true_class = y_test[i]
```

```

# Compute log-likelihood for each class
for c in range(N_classes):
    LL_scores_test[c] = models[c].Viterbi(observation_sequence)

predicted_class = np.argmax(LL_scores_test)

true_labels.append(true_class)
predicted_labels.append(predicted_class)

if predicted_class == true_class:
    correct_test_characters += 1
else:
    total_test_errors += 1

# Calculate CRR and CER for test data
CRR_test = correct_test_characters / total_test_characters
CER_test = total_test_errors / total_test_characters

print(f"\nMNIST Test Results:")
print(f"Character Recognition Rate (CRR): {CRR_test * 100:.2f}%")
print(f"Character Error Rate (CER): {CER_test * 100:.2f}%")

# Create confusion matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(true_labels, predicted_labels)
print(cm)

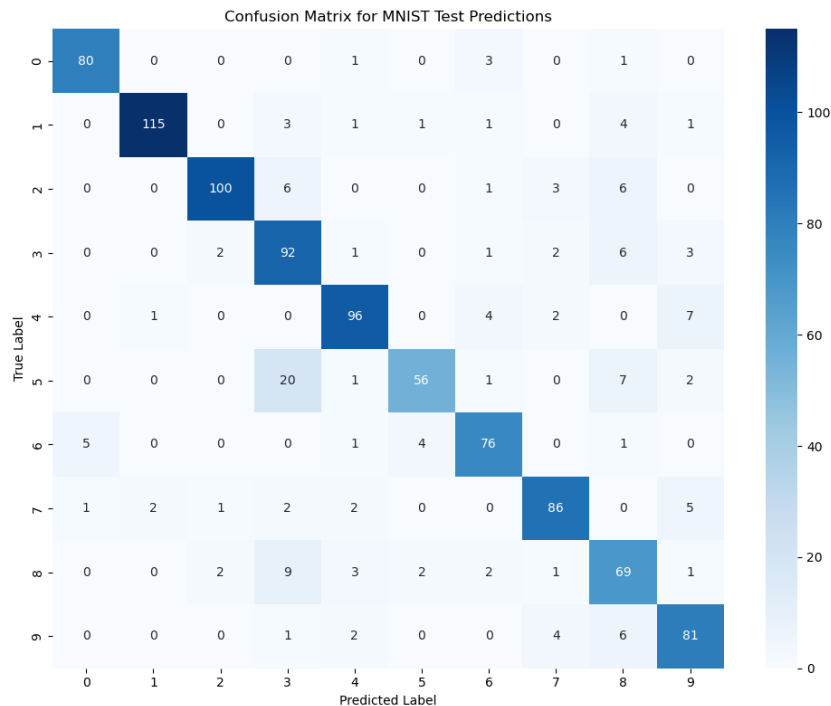
# Plot confusion matrix
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=range(10), yticklabels=range(10))
plt.title('Confusion Matrix for MNIST Test Predictions')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.savefig('confusion_matrix_mnist.png')
plt.show()

# Print classification report
print("\nClassification Report:")
print(classification_report(true_labels, predicted_labels, digits=4))

```

MNIST Test Dataset - Character Recognition Rate (CRR): 11.35%

MNIST Test Dataset - Character Error Rate (CER): 88.65%



We can see that the results of the classification of real data is close to the ones using generated data, this shows that the model is consistent in his classifications. The confusion matrix shows good performance, with most digits being recognized correctly. The main confusions appear to be between similar-looking digits (4 and 9 for example), which is expected.

b. Is Viterbi a good classification method ?

Based on the experimental results obtained from both synthetic and real MNIST test data, the Viterbi algorithm demonstrates strong performance as a classification method for digit recognition using Hidden Markov Models (HMMs), but modern deep learning approaches may achieve higher accuracy rates.

Also, Viterbi's performance is heavily dependent on the quality of the underlying HMM parameters. Poorly trained models will yield suboptimal classification regardless of the decoding algorithm.